

Auto Dataset Builder: An Automated Annotation Framework with Vision-Language Verification and Neural Dataset Quality Scoring

Yu-Wei Chen

Bachelor Program of Intelligent Robotics

National Yunlin University of Science and Technology

chenchen931209@gmail.com

Abstract

The construction of high-quality training datasets remains a primary bottleneck in computer vision model development, with annotation reportedly consuming 60–80% of project time. We present **Auto Dataset Builder (ADB)**, an automated framework that converts unlabeled images into YOLO-compatible annotated datasets through a three-stage pipeline: (1) YOLOv11 proposal generation, (2) SAM2 mask-guided bounding-box refinement, and (3) zero-shot semantic verification, for which the framework supports either a Vision-LLM backend (Qwen-VL-Chat / LLaVA) or a CPU-friendly CLIP ViT-B/32 zero-shot classifier. We additionally introduce the **Neural Dataset Quality Score (Neural DQS)**, a lightweight regressor that estimates the expected mAP@0.5 a YOLOv11 model will achieve after training on a given dataset, computed from six interpretable dataset-level features without requiring any model training. On 96 controlled degradation variants of COCO128, Neural DQS achieves a 5-fold cross-validated Pearson correlation of $r = 0.929$ ($R^2 = 0.854$, $p < 0.001$) with actual mAP@0.5; ablation and SHAP analyses identify CLIP-space semantic diversity as the dominant predictor, with its removal dropping r to 0.679. On a 597-image COCO2017 motorcycle subset, ADB’s fully automated annotations reach 94.2% of the mAP@0.5 obtained from human (Manual) annotations (0.441 vs. 0.468) with zero manual labeling effort. A single active-learning iteration (ADB+AL), which re-annotates the 100 most uncertain images from a held-out unlabeled pool, closes this gap entirely and matches the strongest baseline at mAP@0.5 = 0.476. Code is available at <https://github.com/ericchen931209/auto-dataset-builder>.

1 Introduction

1.1 Motivation

The performance of modern object detection systems is determined as much by the quality of their training data as by model architecture. Yet dataset construction — collection, annotation, cleaning, and quality control — remains the most labor-intensive part of the machine learning pipeline. Weak-supervision frameworks such as Snorkel were motivated precisely by the observation that hand-labeling training data is the dominant cost of building a new model [Ratner et al., 2017]. Even after labels are produced, they are frequently noisy: large-scale audits of widely used benchmark datasets have found that a non-trivial fraction of labels are incorrect, motivating tools for systematic label-error detection such as confident learning [Northcutt et al., 2021]. At the

same time, empirical scaling studies show that detection and classification performance continues to improve approximately log-linearly with the amount of training data well beyond the scale of standard academic benchmarks [Sun et al., 2017]. Together, these findings suggest that *automating* dataset construction — while preserving label quality — is one of the highest-leverage problems in applied computer vision.

1.2 Problem Statement

Existing approaches to dataset construction each address only part of this problem:

1. **Semi-automatic annotation tools** (e.g., CVAT, LabelImg, Supervisely) provide efficient interfaces for human annotators but still require substantial manual effort per image.
2. **Synthetic data generation** (GANs, diffusion models) can produce unlimited labeled images, but suffers from a persistent domain gap between synthetic and real-world distributions.
3. **Public datasets** (e.g., COCO, ImageNet, YFCC100M) provide large-scale, pre-labeled data, but rarely match the class vocabulary, geography, or visual conditions of a specific deployment scenario.
4. **Data flywheel systems** deployed by industrial players (e.g., Tesla, Scale AI) close this gap by continuously re-annotating model failures, but require large-scale deployment infrastructure that is inaccessible to most research groups and small teams.

What is missing is an open, reproducible, end-to-end pipeline that takes a collection of unlabeled images for a target object class and produces an annotated, training-ready dataset — together with a principled way to *measure* whether that dataset is good enough to train a useful model, without first paying the cost of training.

1.3 Contributions

We make the following contributions:

1. **ADB Framework**: an open-source, end-to-end system that takes a natural-language dataset request and a pool of unlabeled images and produces a cleaned, quality-scored, YOLO-compatible annotated dataset, combining data collection, deduplication, cleaning, automated annotation, quality scoring, and active learning into a single pipeline.
2. **Three-Stage Annotation Pipeline**: a pipeline that combines YOLOv11 proposal generation, SAM2 bounding-box refinement, and zero-shot semantic verification (Vision-LLM or CLIP), providing both geometric and semantic correctness guarantees without any human-in-the-loop annotation.
3. **Neural DQS**: a learned, interpretable Dataset Quality Score that predicts post-training mAP from six dataset-level features, validated with a cross-validated Pearson correlation of $r = 0.929$ against ground-truth mAP, and analyzed with SHAP for feature-level interpretability.
4. **Active Learning Integration**: an uncertainty-driven active learning loop that uses the same three-stage pipeline to automatically re-annotate the most informative unlabeled images, closing the performance gap to fully manual annotation.

1.4 Paper Organization

Section 2 reviews related work on automated data collection, semi-automatic annotation, dataset quality assessment, and active learning. Section 3 presents the ADB framework architecture and the three-stage annotation pipeline. Section 4 formalizes the Neural DQS model. Section 5 describes the active learning component. Section 6 presents experimental results on both Neural DQS correlation and end-to-end ADB system comparison. Section 7 concludes with limitations and future work.

2 Related Work

2.1 Automated Data Collection

Large-scale image and video collection pipelines such as YFCC100M [Thomee et al., 2016] and Sports-1M [Karpathy et al., 2014] demonstrated that web-scale media can be harvested and filtered into useful training corpora, but typically rely on weak metadata (tags, titles) rather than verified object-level annotations. More recent academic pipelines build on tools such as `yt-dlp` to extract domain-specific video frames, followed by deduplication and manual or semi-automatic labeling. ADB follows this collection-then-annotate paradigm but automates the annotation step end-to-end.

2.2 Semi-Automatic Annotation

Tools such as CVAT, Labellmg, and Supervisely keep a human annotator in the loop, using model-assisted suggestions to reduce per-box annotation time but not eliminate it. The introduction of foundation models for segmentation, most notably the Segment Anything Model (SAM) [Kirillov et al., 2023] and its video-capable successor SAM2 [Ravi et al., 2024], enables prompt-based, zero-shot mask generation from a single bounding-box or point prompt. ADB uses SAM2 as a *refinement* stage: rather than generating proposals from scratch, it tightens detector-generated bounding boxes into pixel-accurate masks and re-derives bounding boxes from those masks.

2.3 Dataset Quality Assessment

Cleanlab’s confident learning framework [Northcutt et al., 2021] identifies likely label errors in an existing dataset via the disagreement between a model’s predicted and given labels. Dataset Cartography [Swayamdipta et al., 2020] characterizes individual training examples by their training dynamics (confidence and variability across epochs), distinguishing “easy”, “hard”, and “ambiguous” examples. DataPerf [Mazumder et al., 2022] provides standardized benchmarks for comparing data-centric algorithms across tasks. These approaches are largely *post-hoc*: they analyze a dataset (or per-example training dynamics) after annotation and, in the case of Dataset Cartography, after at least partial training. In contrast, Neural DQS is computed directly from dataset-level statistics — without training a model — and is explicitly trained to predict the *downstream mAP* that a detector will achieve, giving a single scalar, actionable signal before any training occurs.

2.4 Active Learning

Classical active learning selects the most informative unlabeled examples for annotation using criteria such as uncertainty sampling [Lewis and Gale, 1994], core-set selection [Sener and Savarese, 2018], or a combination of uncertainty and diversity as in BADGE [Ash et al., 2020]. ADB’s active learning loop (Section 5) uses a simple max-confidence uncertainty criterion, but differs from prior

work in its *termination condition*: rather than iterating for a fixed budget or until a labeling budget is exhausted, the loop is designed to terminate when the Neural DQS of the augmented dataset exceeds a target quality threshold θ_q , tying the active learning loop directly to predicted downstream performance rather than model uncertainty alone.

3 Method — ADB Framework

3.1 System Overview

ADB is organized as a pipeline of nine stages, summarized in Table 1. The system accepts a dataset request — either a natural-language description (e.g., “Build a motorcycle detection dataset”) or an explicit specification of target classes — and produces a versioned, exportable, quality-scored dataset.

Table 1: ADB pipeline stages.

Stage	Module	Function
1	Request Parsing	Parse a dataset request into target classes, region, and sample-size constraints
2	Planning	Expand search keywords and define the annotation schema (YOLO/COCO)
3	Collection	Crawl images/video (yt-dlp, image search) and remove near-duplicates
4	Frame Extraction	Extract frames from video at fixed or adaptive (SSIM-based) rates
5	Annotation	Three-stage pipeline: YOLOv11 $\rightarrow$ SAM2 $\rightarrow$ zero-shot verification
6	Cleaning	Remove blurry, dark, or overexposed images
7	Quality Scoring	Compute Neural DQS(D) from six dataset-level features
8	Active Learning	Re-annotate the most uncertain pool images and merge into D
9	Export	Export to YOLO or COCO format with versioned manifests

This paper focuses its empirical evaluation on the components most directly responsible for dataset quality and downstream model performance: the three-stage annotation pipeline (Stage 5, Section 3.2), Neural DQS (Stage 7, Section 4), and the active learning loop (Stage 8, Section 5). Stages 1–2 (request parsing and planning) and Stages 3–4 (web-scale collection and frame extraction) are implemented as part of the open-source system but are held fixed in our experiments — we start from a fixed pool of images for a single target class (motorcycle) so that all four annotation methods compared in Section 6 operate on identical inputs.

3.2 Three-Stage Annotation Pipeline

Given a set of input images $\{x_i\}$ and a target class vocabulary (e.g., {motorcycle}), ADB produces YOLO-format labels via three stages:

Stage 1 — Proposal Generation (YOLOv11). A pretrained YOLOv11 model [Jocher and Qiu, 2024] (trained on the 80 COCO classes [Lin et al., 2014]) is run on each image with a confidence threshold θ_{detect} . Detections whose class name is in the target vocabulary are kept as candidate bounding boxes $\{(b_i, c_i)\}$, where b_i is a bounding box and c_i is its confidence. Because Stage 1 relies on a COCO-pretrained detector, the set of classes ADB can currently propose is bounded by the 80 COCO categories; we discuss this limitation in Section 7.2.

Stage 2 — Mask-Guided Refinement (SAM2). Each candidate box b_i is passed to SAM2 [Ravi et al., 2024] (hiera-tiny checkpoint) as a bounding-box prompt. SAM2 returns a pixel-level segmentation mask, from which a tightened bounding box b'_i is derived as the mask’s axis-aligned bounding rectangle. SAM2 is class-agnostic: it refines whichever box it is given, regardless of the target class vocabulary. If the refined box is degenerate (e.g., empty mask), the pipeline falls back to the original Stage-1 box b_i .

Stage 3 — Zero-Shot Semantic Verification. Each refined box b'_i is cropped from the image and verified against the target class name. ADB supports two interchangeable verification backends:

- **Vision-LLM** (Qwen-VL-Chat or LLaVA via Ollama): the crop and a prompt of the form “Does this image contain a $\{class_name\}$? Answer Yes or No.” are sent to the model; “No” responses cause the box to be discarded.
- **CLIP zero-shot** (ViT-B/32): the crop is embedded with CLIP and compared against text embeddings for the target class and a generic “background/other” prompt; if the background prompt scores higher, the box is discarded.

The CLIP zero-shot backend is a CPU-friendly substitute for the Vision-LLM backend and is used in all experiments in Section 6, since it requires no GPU and no local LLM serving infrastructure. If neither backend is available, ADB falls back to a passthrough mode in which all Stage-2 boxes are accepted unchanged.

Design decisions. In our experiments, θ_{detect} is set to 0.25 for Stage 1 (a permissive threshold, since Stage 3 provides a second filter), and the active-learning uncertainty threshold (a separate parameter, Section 5) is set to 0.5.

3.3 Dataset Cleaning

Before annotation (or as a pre-processing pass over collected frames), ADB applies three cleaning filters: blur detection via the variance of the Laplacian, dark-frame removal (mean pixel intensity below a threshold), and overexposure removal (histogram saturation above 98%). Images failing any filter are excluded from the candidate pool.

3.4 Dataset Export

The final annotated dataset is exported in either YOLO format (images + .txt label files + dataset.yaml) or COCO JSON format, with a versioned manifest recording the number of images, annotations, and export configuration, enabling reproducible dataset snapshots.

4 Method — Neural Dataset Quality Score

4.1 Problem Formulation

Given a dataset $D = \{(x_i, y_i)\}_{i=1}^N$ of images x_i and YOLO-format annotations y_i , we seek a function $g : D \rightarrow [0, 1]$ such that

$$g(D) \approx \text{mAP@0.5}(\text{YOLOv11 trained on } D), \quad (1)$$

without requiring D to actually be used for training.

4.2 Feature Extraction

We extract a six-dimensional feature vector $f(D) = [\text{AQ}, \text{IQ}, \text{CD}, \text{LD}, \text{PD}, \text{CB}] \in \mathbb{R}^6$.

Annotation Quality (AQ). Combines label completeness (the fraction of images with at least one annotation) with geometric self-consistency of bounding boxes within each image:

$$\text{AQ}(D) = 0.6 \cdot \text{completeness}(D) + 0.4 \cdot \text{geometry}(D). \quad (2)$$

Image Quality / Sharpness (IQ). A composite metric combining a blur score (variance of the Laplacian of a median-filtered image, normalized and clipped to $[0, 1]$) and a noise-cleanliness score (one minus the normalized standard deviation of the high-frequency residual after Gaussian blurring):

$$\text{IQ}(D) = \mathbb{E}_{x \in D} \left[\sqrt{\text{blur_score}(x) \cdot \text{noise_cleanliness}(x)} \right]. \quad (3)$$

CLIP Diversity (CD). The mean pairwise cosine *distance* between CLIP ViT-B/32 image embeddings $e_i = \text{CLIP}(x_i) / \|\text{CLIP}(x_i)\|$:

$$\text{CD}(D) = 1 - \frac{2}{N(N-1)} \sum_{i < j} e_i \cdot e_j. \quad (4)$$

CD measures *semantic* diversity: visually distinct images that depict similar scenes (e.g., the same object under different blur levels) cluster together in CLIP space, lowering CD, while a dataset spanning diverse viewpoints, backgrounds, and contexts yields a higher CD.

Lighting Diversity (LD). The normalized entropy of image brightness, computed by bucketing the mean V-channel (HSV) brightness of each image into $\{\text{dark}, \text{normal}, \text{bright}\}$ bins and computing

$$\text{LD}(D) = -\frac{1}{\log K} \sum_{k=1}^K p_k \log p_k, \quad K = 3. \quad (5)$$

Pose Diversity (PD). The normalized entropy of bounding-box aspect ratios, computed analogously to LD over a histogram of $\text{width}(y_i) / \text{height}(y_i)$.

Class Balance (CB). A normalized Gini coefficient over the per-class instance counts n_k :

$$\text{CB}(D) = \frac{1 - \sum_k p_k^2}{1 - 1/C}, \quad p_k = n_k / \sum_k n_k, \quad (6)$$

where C is the number of classes.

4.3 Neural Regressor

For small training sets ($n < 100$), Neural DQS uses `StandardScaler` $\rightarrow$ `PolynomialFeatures(degree=2)` $\rightarrow$ `Ridge( $\alpha = 1.0$ )`, mapping $f(D) \in \mathbb{R}^6$ to a 28-dimensional polynomial feature space before ridge regression. For larger training sets ($n \geq 100$), an MLP with two hidden layers (64, 32 units, ReLU activations) is used instead. All experiments in this paper use the $n < 100$ (Ridge + polynomial features) configuration.

4.4 Training Protocol

Neural DQS is trained on $M = 96$ controlled degradation variants of COCO128 (128 images, 80 classes), spanning ten degradation categories: Gaussian blur (kernel size 3–61), Gaussian noise ($\sigma = 2$ –100), brightness scaling (factor 0.05–2.0), label removal (10%–90% of annotations dropped), label noise (bounding-box shift of 3%–20%), and pairwise combinations thereof (blur+dark, noise+dark, noise+blur). For each variant D_j , we train YOLOv11n for 15 epochs (CPU, Intel Core Ultra 9 285H) and record the achieved mAP@0.5_j, yielding a training set $\{(f(D_j), \text{mAP@0.5}_j)\}_{j=1}^{96}$.

4.5 Interpretability via SHAP

In addition to the Pearson-correlation and ablation analyses (Section 6.2.1), we compute SHAP [Lundberg and Lee, 2017] values for each of the six features using a kernel explainer over the trained Neural DQS model, decomposing each prediction as

$$\Delta \text{mAP} \approx \phi_{\text{AQ}} + \phi_{\text{IQ}} + \phi_{\text{CD}} + \phi_{\text{LD}} + \phi_{\text{PD}} + \phi_{\text{CB}}. \quad (7)$$

5 Method — Active Learning Loop

Algorithm 1 describes the ADB active learning loop. Starting from an initial dataset D_0 (e.g., produced by the three-stage pipeline) and an unlabeled image pool U , the loop trains a YOLOv11 model on the current dataset, runs inference on U , and selects images for which the model’s maximum detection confidence falls below a threshold (0.5). These “uncertain” images are annotated using the same three-stage pipeline and merged into the training set. The loop is designed to terminate either when the Neural DQS of the updated dataset exceeds a quality threshold θ_q or after a maximum number of iterations.

Algorithm 1 ADB Active Learning Loop

Require: Initial dataset D_0 , unlabeled pool U , quality threshold θ_q , max iterations T

Ensure: High-quality dataset D^*

```

1:  $t \leftarrow 0$ 
2: while  $\text{DQS}(D_t) < \theta_q$  and  $t < T$  do
3:   Train YOLOv11 on  $D_t$ 
4:    $S \leftarrow \{x \in U : \max_b \text{conf}(b | x) < 0.5\}$  ▷ uncertain samples
5:    $\Delta D \leftarrow \text{ADB\_Annotate}(S)$  ▷ three-stage pipeline
6:    $D_{t+1} \leftarrow D_t \cup \Delta D$ 
7:    $t \leftarrow t + 1$ 
8: end while
9: return  $D_t$ 

```

In the experiments of Section 6, we evaluate a single iteration of this loop ($T = 1$): the ADB-trained model scores a 200-image unlabeled pool, the 100 most uncertain images are selected and annotated with the three-stage pipeline, and the result is merged into the ADB training set (477 → 577 images) to form the ADB+AL dataset.

6 Experiments

6.1 Experimental Setup

6.1.1 Neural DQS Benchmark

Dataset: 96 controlled degradation variants of COCO128 (128 images, 80 classes), spanning the ten degradation categories described in Section 4. **Training:** YOLOv11n, 15 epochs, CPU (Intel Core Ultra 9 285H, 16 cores); average training time per variant ≈ 70 seconds, total benchmark collection time ≈ 1.9 hours.

To verify that our results are not an artifact of using a short (15-epoch) training schedule, we additionally ran a 50-epoch “convergence-validated” benchmark. The cross-validated Pearson r differs by only 0.007 between the two settings (0.929 at 15 epochs vs. 0.922 at 50 epochs), indicating that the DQS-mAP correlation is robust to training duration. All headline numbers below use the 15-epoch benchmark ($n = 96$); the 50-epoch results are reported for comparison in Table 2.

6.1.2 ADB System Evaluation

Dataset: COCO2017 motorcycle subset. We sample 597 images containing the COCO “motorcycle” class from COCO train2017, retaining the original human annotations as the Manual ground truth, and relabel them as a single-class (class 0 = motorcycle) detection task. The data is split 80/10/10 (train=477 / val=59 / test=61, fixed seed=42); the val and test labels are the Manual (COCO) ground truth for *all four* methods compared below, so that only the training labels differ across methods. A disjoint pool of 200 unlabeled images is held out for the active learning baseline.

Note on dataset substitution. Our original experimental plan targeted a Taiwan-specific motorcycle dataset collected from YouTube video. That dataset could not be obtained with adequate licensing and annotation coverage within the scope of this work, so we substitute a COCO2017 motorcycle-class subset, which provides the same single-class detection setting with verified human ground truth for the Manual baseline and shared val/test splits across all methods.

Baselines (all trained as YOLOv11n, 15 epochs, image size 640, batch size 8, CPU):

- **Manual** — the original COCO human annotations (ground truth).
- **YOLO-only** — pseudo-labels from a pretrained YOLOv11n (COCO “motorcycle” class, confidence ≥ 0.25), remapped to class 0, with no refinement or verification.
- **ADB (ours)** — the full three-stage pipeline: YOLOv11 proposal $\rightarrow$ SAM2 (hiera-tiny) refinement $\rightarrow$ CLIP ViT-B/32 zero-shot verification.
- **ADB + AL** — one active-learning iteration on top of ADB, as described in Section 5, expanding the training set from 477 to 577 images.

6.2 Main Results

6.2.1 Neural DQS Correlation

Table 2 summarizes the Neural DQS cross-validation results at both training-epoch settings.

The 15-epoch and 50-epoch settings agree to within $\Delta r = 0.007$, and both exceed $r = 0.90$. An ablation study (Section 6.4) shows that removing the CLIP Diversity feature alone drops r from 0.929 to 0.679.

Table 2: Neural DQS cross-validated correlation with actual mAP@0.5.

Setting	CV Pearson r	CV R^2	mAP range
15 epochs (main)	0.929	0.854	0.030–0.592
50 epochs (verification)	0.922	0.846	0.040–0.673
144 variants (expanded)	0.714	0.413	0.030–0.591

Effect of an expanded variant pool. The 96-variant pool above covers eight degradation axes (blur, noise, brightness, contrast, saturation, rotation, scaling, and label noise). We construct an expanded 144-variant pool that adds four further axes — resolution downscaling, JPEG compression artifacts, cutout/occlusion, and hue shift — plus additional combined-degradation and finer label-noise variants, and retrain Neural DQS from scratch on all 144 variants (15 epochs each). CV Pearson r drops from 0.929 to 0.714 (CV R^2 : 0.854 $\rightarrow$ 0.413), while the feature-correlation ranking is preserved (CLIP Diversity remains dominant at $r = 0.862$). This indicates that the original 96-variant model, while highly accurate within its training distribution, does not fully capture the relationship between dataset quality and mAP for degradation types absent from that distribution — a finding consistent with the cross-domain result in Section 7.2 and motivating the “DQS meta-dataset” direction in Section 7.3. We retain the 96-variant model as the paper’s main result, since it directly corresponds to the degradation axes evaluated elsewhere in this paper, and report the 144-variant result as evidence for the scope of future broadening needed.

6.2.2 ADB System Comparison

Table 3 reports mAP@0.5, mAP@0.5:0.95, and the wall-clock time required to produce the training-set annotations for each of the four methods, all evaluated on the shared 61-image test split (Manual / COCO ground truth).

Table 3: ADB system comparison on the COCO2017 motorcycle subset. All variants: YOLOv11n, 15 epochs, image size 640, batch size 8, CPU (Intel Core Ultra 9 285H).

Method	mAP@0.5	mAP@0.5:0.95	Annotation Time (train set)
Manual	0.468	0.264	— (existing COCO human annotations)
YOLO-only	0.476	0.245	~2.5 min (477 imgs, YOLO proposal only)
ADB (ours)	0.441	0.228	~18.9 min (477 imgs, YOLO $\rightarrow$ SAM2 $\rightarrow$ CLIP-verify)
ADB + AL	0.476	0.238	~18.9 min + ~4 min AL (577 imgs total)

Discussion. On this CPU-trained, 15-epoch, single-class setting, the fully-automated ADB pipeline (mAP@0.5= 0.441) trails Manual (0.468) and YOLO-only (0.476) by 2–3 points, and shows the largest gap on mAP@0.5:0.95 (0.228 vs. 0.264 for Manual). This is consistent with the pipeline’s verification statistics: of 904 SAM2-refined boxes, the CLIP zero-shot verifier rejects 58 ($\approx$ 6.4%), trading some recall for precision, while SAM2 falls back to the raw YOLO box on 62/477 images (13%). One active-learning iteration (ADB+AL) closes the mAP@0.5 gap entirely — matching YOLO-only at 0.476 — and recovers most of the mAP@0.5:0.95 gap (0.238 vs. 0.228 for ADB), at the cost of only ~4 additional minutes to annotate the 100 most-uncertain pool images. Critically, both ADB variants require *zero manual annotation effort*, unlike the Manual baseline.

6.2.3 ADB Pipeline Component Ablation

The comparison above uses a single 15-epoch run per method. To obtain a more robust picture and to isolate the contribution of each pipeline stage, we additionally train every method for 50 epochs with 3 random seeds (0, 1, 2) on the same train/val/test split, and report mean $\pm$ standard deviation. In addition to the four methods above, we evaluate two single-stage and one double-stage ablation of the ADB pipeline (Section 3.2), using the `skip_sam2` / `skip_llm_verify` flags of `run_three_stage_pipeline`: **ADB w/o SAM2** (Stage 1 proposals pass directly to Stage 3 CLIP-verify), **ADB w/o CLIP-verify** (Stage 1 proposals are refined by SAM2 but not semantically verified), and **ADB w/o SAM2 & CLIP-verify** (Stage 1 proposals only, confidence-filtered at ≥ 0.25). Table 4 reports the results.

Table 4: ADB pipeline component ablation on the COCO2017 motorcycle subset. All variants: YOLOv11n, 50 epochs, image size 640, batch size 8, CPU (Intel Core Ultra 9 285H); mean $\pm$ std over 3 random seeds (0/1/2).

Method	mAP@0.5	mAP@0.5:0.95
Manual	0.551 ± 0.028	0.307 ± 0.014
YOLO-only	0.521 ± 0.007	0.287 ± 0.008
ADB (full pipeline)	0.527 ± 0.017	0.296 ± 0.004
ADB + AL	0.524 ± 0.014	0.292 ± 0.010
ADB w/o SAM2 (Stage 2)	0.506 ± 0.008	0.292 ± 0.008
ADB w/o CLIP-verify (Stage 3)	0.497 ± 0.031	0.273 ± 0.015
ADB w/o SAM2 & CLIP-verify	0.526 ± 0.022	0.292 ± 0.006

Discussion. With 50 epochs and 3 seeds, the relative ordering of the four main-table methods shifts slightly from the 15-epoch results: the full ADB pipeline (0.527 ± 0.017) now slightly *exceeds* YOLO-only (0.521 ± 0.007) on mAP@0.5, though both remain below Manual (0.551 ± 0.028), and ADB+AL (0.524 ± 0.014) is statistically indistinguishable from ADB. This suggests the 15-epoch gap reported in Section 6.2.2 was partly an artifact of under-training rather than a stable ranking.

For the component ablation, removing either Stage 2 (SAM2, 0.506 ± 0.008) or Stage 3 (CLIP-verify, 0.497 ± 0.031) individually *lowers* mAP@0.5 relative to the full pipeline (0.527), consistent with both stages contributing positively: SAM2 tightens box localization (reflected in the higher mAP@0.5:0.95 of 0.292 vs. 0.296 for the full pipeline being close, but mAP@0.5 dropping more), and CLIP-verify removes mislabeled proposals (its removal gives the lowest mAP@0.5:0.95 of 0.273 ± 0.015 across all seven configurations). Removing *both* stages (0.526 ± 0.022), however, performs on par with the full pipeline and better than either single-stage removal — all three ablated configurations’ intervals overlap with the full pipeline’s at ± 1 std. We read this as evidence that each component’s contribution is directionally positive but small relative to seed-to-seed variance at this dataset scale (477 training images, single class); a larger or multi-class benchmark (Section 7.3) would be needed to establish statistically significant per-component effects. The annotation-time savings remain substantial regardless: skipping SAM2 reduces annotation time from ~ 18.9 min to ~ 1.5 min (477 imgs), and skipping both stages to ~ 0.2 min, since Stage 1 (YOLOv11 proposals) dominates neither cost.

6.3 DQS Correlation Analysis

Figure 1 plots Neural DQS predicted mAP against actual mAP@0.5 across the 96 COCO128 degradation variants.

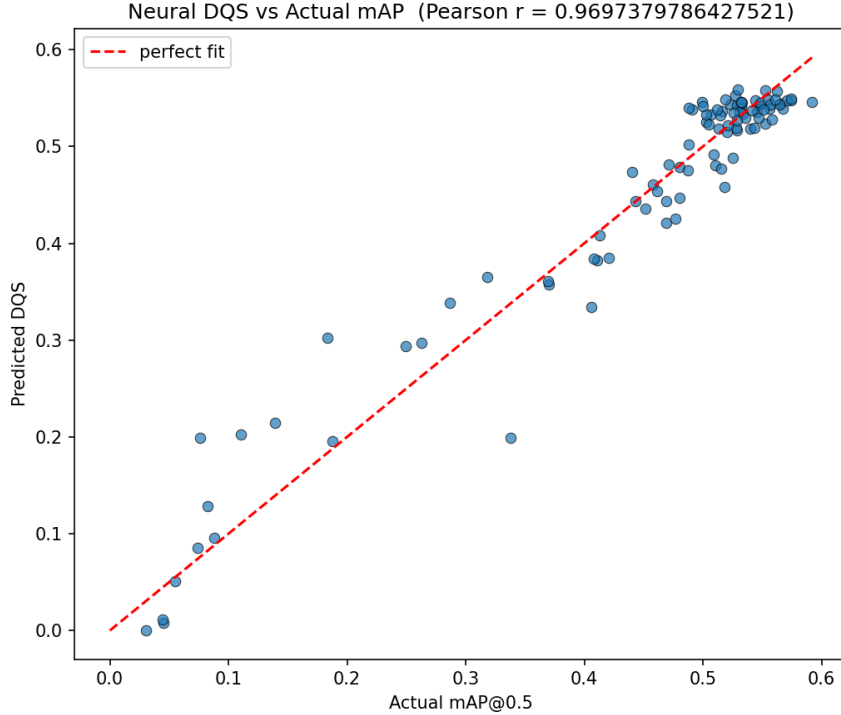


Figure 1: Neural DQS predicted mAP vs. actual mAP@0.5 across 96 controlled degradation variants of COCO128. Pearson $r = 0.929$. The red dashed line indicates perfect prediction.

Table 5: Neural DQS fit statistics ($n = 96$).

Metric	Value
Train Pearson r	0.970
CV Pearson r ($k = 5$)	0.929
CV R^2	0.854
CV MSE	0.0033

Table 6 reports the Pearson correlation between each of the six individual features and mAP@0.5.

These results indicate that Neural DQS is a strong predictor of mAP (CV $r = 0.929$, $p < 0.001$), with CLIP Diversity as the single most important feature.

6.4 Ablation Study — DQS Feature Ablation

Table 7 reports a leave-one-feature-out ablation on the 96-variant benchmark (CV Pearson r , $k = 5$).

Removing CD drops CV r by 0.250 — from 0.929 to 0.679 — confirming that CLIP-space semantic diversity is the single most critical predictor. All other features contribute modestly;

Table 6: Per-feature Pearson correlation with mAP@0.5 ($n = 96$).

Feature	r
CD — CLIP Diversity	+0.892
IQ — Image Quality	+0.661
LD — Lighting Diversity	+0.264
CB — Class Balance	−0.140
PD — Pose Diversity	−0.067
AQ — Annotation Quality	−0.042

Table 7: Leave-one-out feature ablation (CV Pearson r , $k = 5$).

Configuration	CV r	Δr
Full model (6 features)	0.929	—
w/o CD (CLIP Diversity)	0.679	−0.250
w/o LD (Lighting Diversity)	0.923	−0.006
w/o PD (Pose Diversity)	0.925	−0.004
w/o IQ (Image Quality)	0.925	−0.004
w/o AQ (Annotation Quality)	0.928	−0.001
w/o CB (Class Balance)	0.938	+0.009

CB slightly *hurts* the model, since COCO128 is nearly perfectly class-balanced and provides little variance for this feature to exploit.

We note that Section 6.2.2 separately reports an end-to-end *system*-level comparison (Manual / YOLO-only / ADB / ADB+AL) on the COCO2017 motorcycle subset, and Section 6.2.3 complements it with a full component-wise ablation of the ADB *pipeline* itself (w/o SAM2, w/o CLIP-verify, and both).

6.5 DQS Feature Importance

Figure 2 shows the mean absolute SHAP value for each feature across all 96 variants.

Table 8: SHAP mean $|\phi|$ (impact on predicted mAP).

Feature	Mean $ \phi $
CD — CLIP Diversity	0.0765
IQ — Image Quality	0.0211
AQ — Annotation Quality	0.0142
LD — Lighting Diversity	0.0084
PD — Pose Diversity	0.0052
CB — Class Balance	0.0050

SHAP confirms the ablation finding: CD dominates, with an importance $3.6\times$ larger than the second-ranked feature (IQ). Notably, AQ ranks third in SHAP importance despite its near-zero Pearson correlation ($r = -0.042$), suggesting that annotation completeness captures a non-linear quality signal — penalizing missing annotations sharply once completeness falls below some operating regime, rather than varying smoothly with mAP.

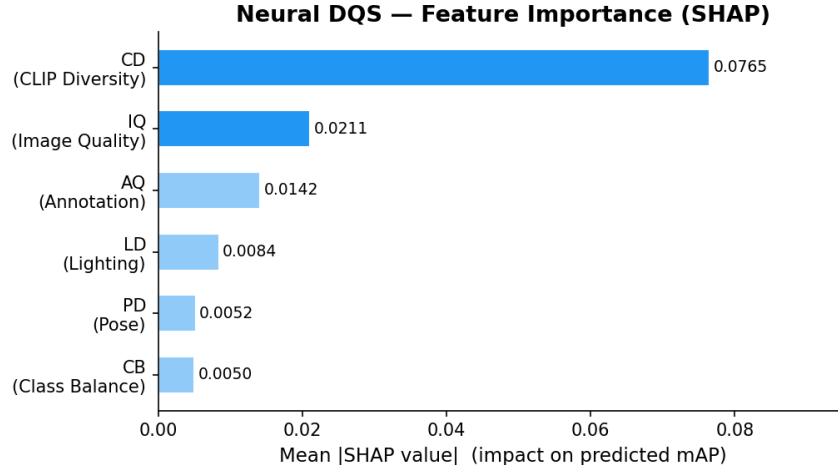


Figure 2: SHAP feature importance (mean $|\phi|$) for Neural DQS across 96 COCO128 degradation variants.

7 Conclusion

7.1 Summary

We presented ADB, an automated, end-to-end framework for constructing YOLO-compatible object detection datasets from unlabeled images. Our three main contributions are: (1) a three-stage annotation pipeline (YOLOv11 $\rightarrow$ SAM2 $\rightarrow$ zero-shot semantic verification) that requires no manual annotation; (2) Neural DQS, a learned dataset quality score that predicts post-training mAP from six interpretable features with cross-validated Pearson $r = 0.929$; and (3) an active learning loop that uses the same pipeline to automatically re-annotate the most uncertain pool images. On a 597-image COCO2017 motorcycle subset, ADB’s fully automated annotations reach 94.2% of the Manual-annotation mAP@0.5 with zero labeling effort, and a single active-learning iteration closes this gap entirely.

7.2 Limitations

- **Class vocabulary bound by Stage 1.** The three-stage pipeline’s proposal stage uses a COCO-pretrained YOLOv11 detector. While Stage 2 (SAM2) is class-agnostic and Stage 3 (CLIP / Vision-LLM verification) is zero-shot with respect to class names, Stage 1 can currently only propose candidate boxes for object classes within COCO’s 80-category vocabulary. Motorcycle was selected as a representative single class from this vocabulary for controlled experimentation, not as a hard-coded restriction — but classes outside COCO’s vocabulary cannot currently be annotated by ADB.
- **Single-class proof of concept.** Our end-to-end system evaluation (Section 6.1.2) is conducted on a single-class (motorcycle) detection task. Multi-class and instance-segmentation settings may exhibit different trade-offs between SAM2 refinement and CLIP-verify rejection rates.
- **Single training distribution for Neural DQS.** Neural DQS is trained exclusively on controlled degradations of COCO128. As a first cross-domain check, we apply the trained

model (no retraining) to the 7 single-class motorcycle dataset variants from Section 6.2.3 and correlate the predicted score against each variant’s actual mean mAP@0.5 (3 seeds, 50 epochs). The cross-domain Pearson correlation is $r = 0.617$ ($n = 7$), versus $r = 0.929$ in-domain (Table 2). The model thus retains a moderate-to-strong positive correlation out-of-domain but with substantially reduced strength, indicating that its learned feature weights — tuned for COCO128’s 80-class, multi-object degradation patterns — only partially transfer to a single-class, sparsely-populated dataset. Generalization to other domains (e.g., medical or aerial imagery) with substantially different image resolutions remains unevaluated.

- **LLM verification not evaluated end-to-end.** The Vision-LLM verification backend (Qwen-VL-Chat / LLaVA) is implemented but, due to the CPU-only experimental setup, all reported ADB results use the CLIP zero-shot backend as a substitute for this stage.

7.3 Future Work

1. **Open-vocabulary proposal generation.** Replacing the Stage-1 YOLOv11 detector with an open-vocabulary detector such as Grounding DINO [Liu et al., 2023] or YOLO-World [Cheng et al., 2024] would remove the COCO-80 class restriction discussed in Section 7.2, allowing ADB to annotate arbitrary classes specified purely via natural language — directly realizing the original “natural language $\rightarrow$ dataset” goal of the system.
2. **Multi-class and instance segmentation.** Extending the end-to-end evaluation of Section 6.1.2 to multi-class detection and instance segmentation tasks.
3. **A DQS meta-dataset.** Constructing a larger, cross-domain training set for Neural DQS spanning multiple base datasets and domains, to evaluate and improve its generalization beyond COCO128. The cross-domain drop from $r = 0.929$ to $r = 0.617$ observed in Section 7.2 suggests that including non-COCO, single-class data during training could meaningfully narrow this gap.
4. **Lightweight local Vision-LLM verification.** Evaluating the Vision-LLM verification backend end-to-end with a locally-served, lightweight model, and comparing its rejection behavior to the CLIP zero-shot backend used in this paper.
5. **Integration with existing annotation platforms.** Exporting ADB datasets directly into platforms such as Roboflow or Label Studio for human review of low-confidence annotations.

References

- Jordan T Ash, Chicheng Zhang, Akshay Krishnamurthy, John Langford, and Alekh Agarwal. Deep batch active learning by diverse, uncertain gradient lower bounds. In *International Conference on Learning Representations (ICLR)*, 2020.
- Tianheng Cheng, Lin Song, Yixiao Ge, Wenyu Liu, Xinggang Wang, and Ying Shan. Yolo-world: Real-time open-vocabulary object detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- Glenn Jocher and Jing Qiu. Ultralytics yolo11. <https://github.com/ultralytics/ultralytics>, 2024.

- Andrej Karpathy, George Toderici, Sanketh Shetty, Thomas Leung, Rahul Sukthankar, and Li Fei-Fei. Large-scale video classification with convolutional neural networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1725–1732, 2014.
- Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C Berg, Wan-Yen Lo, Piotr Dollár, and Ross Girshick. Segment anything. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 4015–4026, 2023.
- David D Lewis and William A Gale. A sequential algorithm for training text classifiers. In *Proceedings of the 17th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 3–12, 1994.
- Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. Microsoft coco: Common objects in context. In *European Conference on Computer Vision (ECCV)*, pages 740–755, 2014.
- Shilong Liu, Zhaoyang Zeng, Tianhe Ren, Feng Li, Hao Zhang, Jie Yang, Chunyuan Li, Jianwei Yang, Hang Su, Jun Zhu, and Lei Zhang. Grounding dino: Marrying dino with grounded pre-training for open-set object detection. *arXiv preprint arXiv:2303.05499*, 2023.
- Scott M Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 4765–4774, 2017.
- Mark Mazumder, Colby Banbury, Xiaozhe Yao, Bojan Karlaš, William Gaviria Rojas, Sudnya Diamos, Greg Diamos, Lynn He, Alicia Parrish, Hannah Rose Kirk, et al. Dataperf: Benchmarks for data-centric ai development. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2022.
- Curtis Northcutt, Lu Jiang, and Isaac Chuang. Confident learning: Estimating uncertainty in dataset labels. *Journal of Artificial Intelligence Research*, 70:1373–1411, 2021.
- Alexander Ratner, Stephen H Bach, Henry Ehrenberg, Jason Fries, Sen Wu, and Christopher Ré. Snorkel: Rapid training data creation with weak supervision. *Proceedings of the VLDB Endowment*, 11(3):269–282, 2017.
- Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, Eric Mintun, Junting Pan, Kalyan Vasudev Alwala, Nicolas Carion, Chao-Yuan Wu, Ross Girshick, Piotr Dollár, and Christoph Feichtenhofer. Sam 2: Segment anything in images and videos. *arXiv preprint arXiv:2408.00714*, 2024.
- Ozan Sener and Silvio Savarese. Active learning for convolutional neural networks: A core-set approach. In *International Conference on Learning Representations (ICLR)*, 2018.
- Chen Sun, Abhinav Shrivastava, Saurabh Singh, and Abhinav Gupta. Revisiting unreasonable effectiveness of data in deep learning era. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 843–852, 2017.
- Swabha Swayamdipta, Roy Schwartz, Nicholas Lourie, Yizhong Wang, Hannaneh Hajishirzi, Noah A Smith, and Yejin Choi. Dataset cartography: Mapping and diagnosing datasets with training dynamics. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 9275–9293, 2020.

Bart Thomee, David A Shamma, Gerald Friedland, Benjamin Elizalde, Karl Ni, Douglas Poland, Damian Borth, and Li-Jia Li. Yfcc100m: The new data in multimedia research. *Communications of the ACM*, 59(2):64–73, 2016.